Enterprise WhatsApp platform architecture

A technical blueprint for a product with capabilities similar to WhatChimp

Enterprise design from day one • 29 September 2026

Build a multi-tenant messaging platform around Meta’s official WhatsApp Business Platform, with regional isolation, durable message processing, a visual automation builder, a shared agent inbox, campaigns, integrations, and controlled AI assistance. The central design principle is to persist intent before executing an external action and preserve enough evidence to recover safely after failure.

Architecture decision

Use bounded regional cells, each spanning three availability zones. A tenant has one home cell and one active write region. Deploy a separate control plane for tenant provisioning, subscriptions, and placement. Keep message processing operational with a cached, signed configuration when the control plane is unavailable. Provide dedicated cells for customers requiring stronger infrastructure isolation.

What this document provides

The following pages connect product capabilities to network routes, services, data stores, events, failure handling, and operational controls. Twelve architecture and flow diagrams show how onboarding, inbound messages, outgoing replies, campaigns, bot execution, AI, and disaster recovery work. API examples, capacity calculations, implementation phases, and acceptance tests make the design actionable for an engineering team.

Scope and assumptions

This is a proposed architecture for a similar product, not a description of WhatChimp’s private implementation. AWS is the concrete infrastructure reference; the boundaries are portable to other clouds. WhatsApp is the primary channel, with adapters for Instagram, Messenger, and web chat. Voice calling and payment-card processing are separate future domains. All capacities and SLOs are design targets to validate under load, not vendor guarantees.

The commercial model assumes tenant-owned WhatsApp accounts and, where available, direct provider billing. Partner eligibility, permissions, rates, throughput entitlements, and regional availability must be confirmed for the chosen launch markets and Meta API version. Source notes and official references appear on pages 31 and 32.

Document map

Read pages 3 to 7 for the overall system. Pages 8 to 20 trace the working pipelines. Pages 21 to 30 define the engineering and operational requirements for an enterprise launch.

1 Product capabilities and system context 3
2 Network topology and trust boundaries 4
3 Connection and protocol inventory 5
4 Regional cells and tenant placement 6
5 Service boundaries and ownership 7
6 Tenant onboarding and channel connection 8
7 Inbound webhook pipeline 9
8 Outbound message pipeline 10
9 Events ordering and reliable state changes 11
10 Core data model 12
11 Storage constraints and retention 13
12 Consent templates and send policy 14
13 Campaign and sequence pipeline 15
14 Visual chatbot builder and publication 16
15 Durable automation runtime 17
16 Shared inbox and realtime collaboration 18
17 AI knowledge and response pipeline 19
18 Integration media and commerce pipelines 20
19 Security and tenant isolation 21
20 Public API and event contracts 22
21 Analytics billing and audit pipelines 23
22 Availability targets and observability 24
23 Capacity model and cost drivers 25
24 Failure handling and disaster recovery 26
25 Delivery pipeline and deployment 27
26 Verification and launch acceptance 28
27 Enterprise implementation sequence 29
28 Decisions to finalize before engineering 30

Terms used throughout

Tenant means a customer organization. Cell means an isolated deployment serving a bounded set of tenants. WABA means WhatsApp Business Account. Outbox means a database record committed with business data and later published as an event. DLQ means a quarantine queue for work that cannot be processed automatically. RPO is the maximum targeted data loss after disaster; RTO is the target time to restore service.

1 Product capabilities and system context

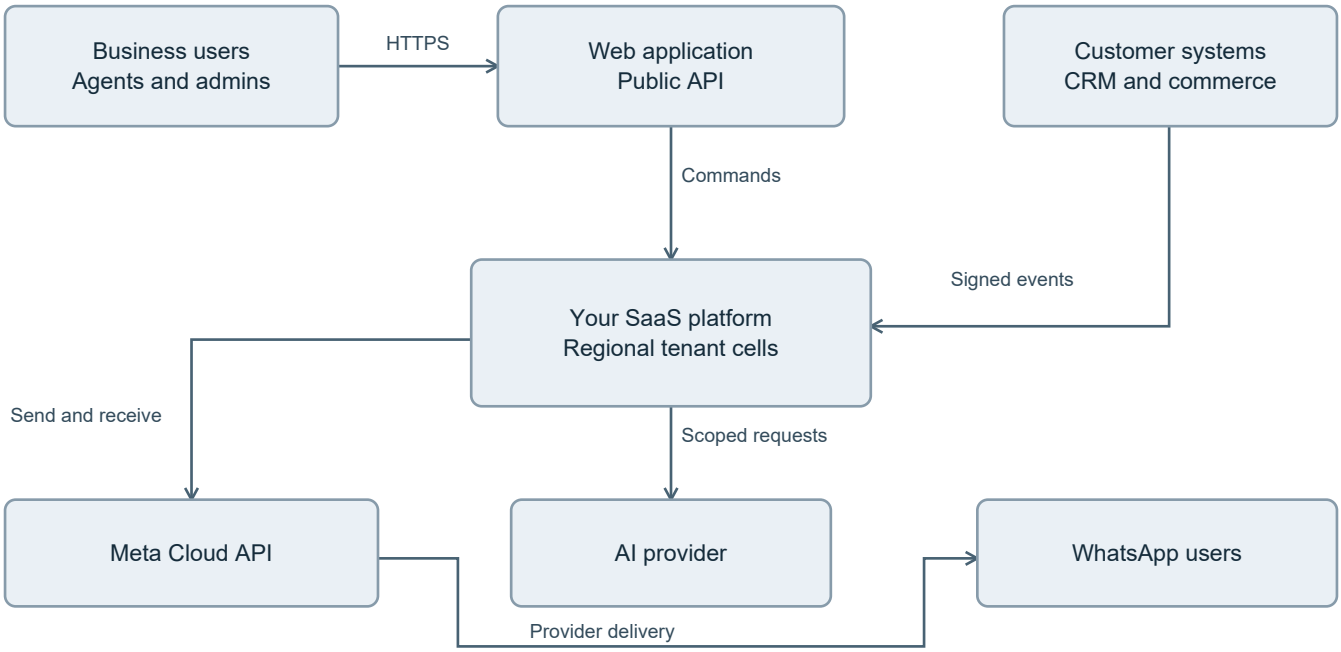


Figure 1. The platform owns business workflows and records; Meta owns delivery over the WhatsApp network. The AI provider is optional in the reply path.

Capability	Services that make it work
Team inbox	Conversation ownership, agent routing, realtime fan-out, message commands
Broadcasts and sequences	Audience snapshots, policy evaluation, scheduling, quotas, outbound dispatch
Visual bots and AI	Flow compiler, durable runtime, knowledge retrieval, tool authorization
CRM and commerce	Contact identity, custom fields, connectors, media, catalogs and webhook delivery
Enterprise administration	SSO, RBAC, audit, tenant placement, billing, retention and exports

WhatChimp publicly describes campaigns, shared inboxes, chatbot flows, AI agents, and integrations. These define the feature reference, while the service decomposition here is a new engineering proposal. [1, 2]

2 Network topology and trust boundaries

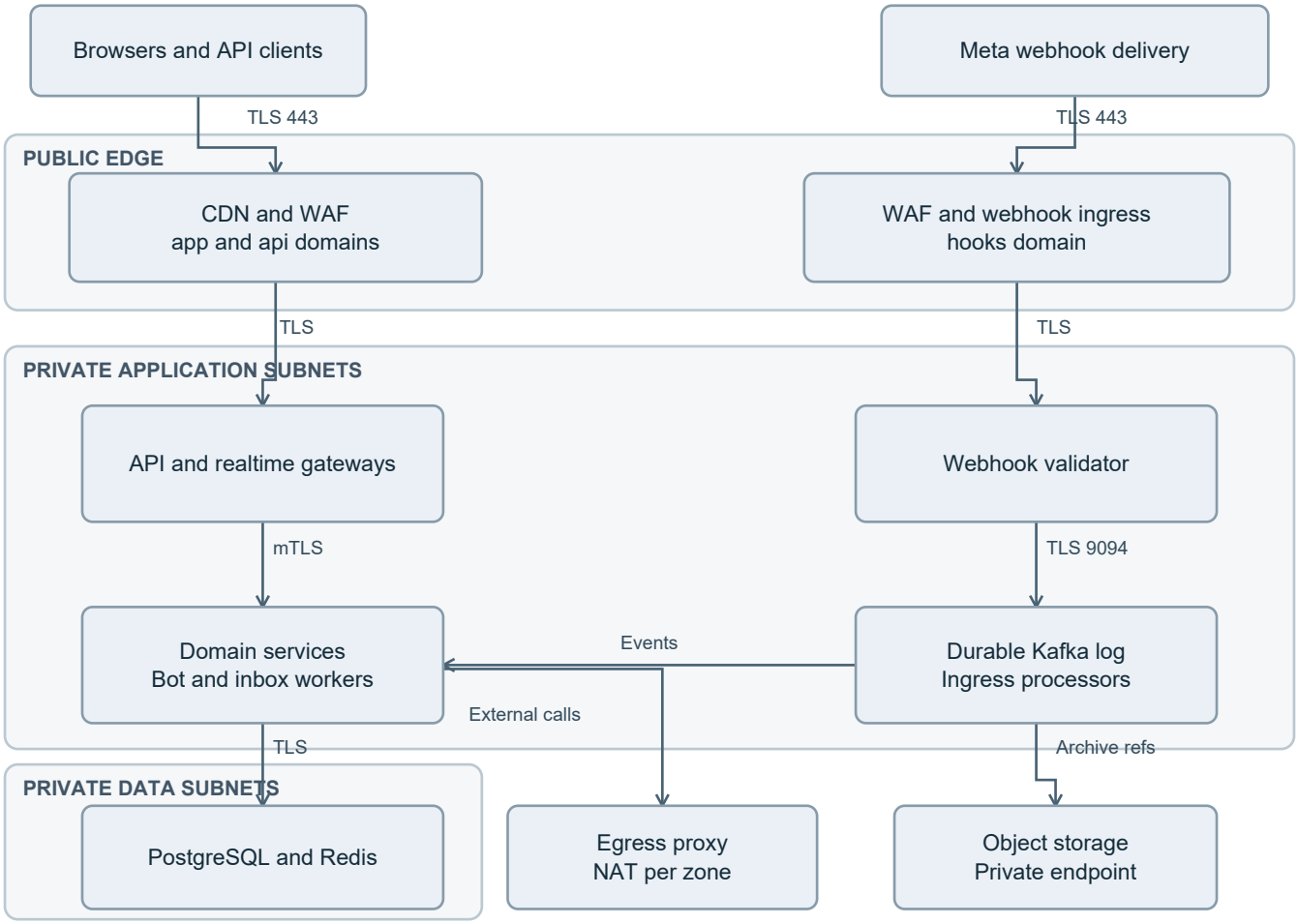


Figure 2. Repeat private application and data capacity across three availability zones. Arrows summarize data paths; connection initiators and allowed protocols are listed on page 5.

Use separate cloud accounts for production, non-production, security logs, and recovery. Route public DNS to managed edge protection; restrict origin access to approved edge paths. Give pods and databases no public IPs. Expose only the API, realtime handshake, provider webhook, and narrowly scoped signup callback routes.

Use private endpoints for object storage, secrets, and supported managed services. Internet-bound API calls leave through an egress proxy and zone-local NAT. Allow named provider destinations through the proxy; plain network policies cannot reliably enforce domain names. NetworkPolicy requires a compatible network plugin and explicit DNS access. [11]

3 Connection and protocol inventory

Initiator and destination	Protocol	Authorization and purpose
Browser → CDN / API edge	HTTPS 443	OIDC session cookie; CSRF protection for changes. Static assets may be public; tenant responses are never shared-cache content.
Browser → realtime gateway	WSS 443	Short-lived, audience-bound connection token; tenant and conversation access checked on subscribe.
Meta → webhook endpoint	HTTPS 443	GET challenge token for setup; POST body HMAC with app secret. Preserve raw bytes until verification.
Edge → gateway / ingress	TLS 443	Origin access control plus service identity; no direct public bypass.
Service → service	mTLS 8443	Workload identity, explicit allowed caller, request deadline, tenant context and trace ID.
Services → PostgreSQL pool	TLS 5432	Per-service role; transaction-scoped tenant context; bounded connection pool.
Services → managed Redis	TLS 6379	Named ACL user; cache, presence and rate counters. Never the only message record.
Services → Kafka brokers	TLS 9094	Concrete reference listener; workload auth and topic ACLs. Port changes with provider authentication mode.
Workers → workflow engine	TLS 7233	Private Temporal frontend; namespace permissions and payload encryption.
Workers → cloud services	HTTPS 443	Private endpoints and workload IAM roles; tenant-scoped object keys.
Egress → Meta / CRM / AI	HTTPS 443	Server-held provider credentials; bounded timeout and destination policy.
Operator → admin plane	HTTPS 443	SSO, MFA, device posture and time-limited role. Database access only through audited private access.
Pods → approved DNS	UDP / TCP 53	Cluster resolver only. Block arbitrary DNS and metadata endpoints.
Services → telemetry collector	mTLS 4317	OTLP telemetry with redaction and workload identity; collector exports over TLS.

All entries are proposed allowed paths. Deny other ingress and egress by default. Set TLS certificate rotation, request/body size limits, connection budgets, and idle timeouts per endpoint. WAF rules for webhooks must not require browser challenges. A provider source-IP allowlist may supplement signature validation only if supported and maintained; it cannot replace it. [6, 11]

4 Regional cells and tenant placement

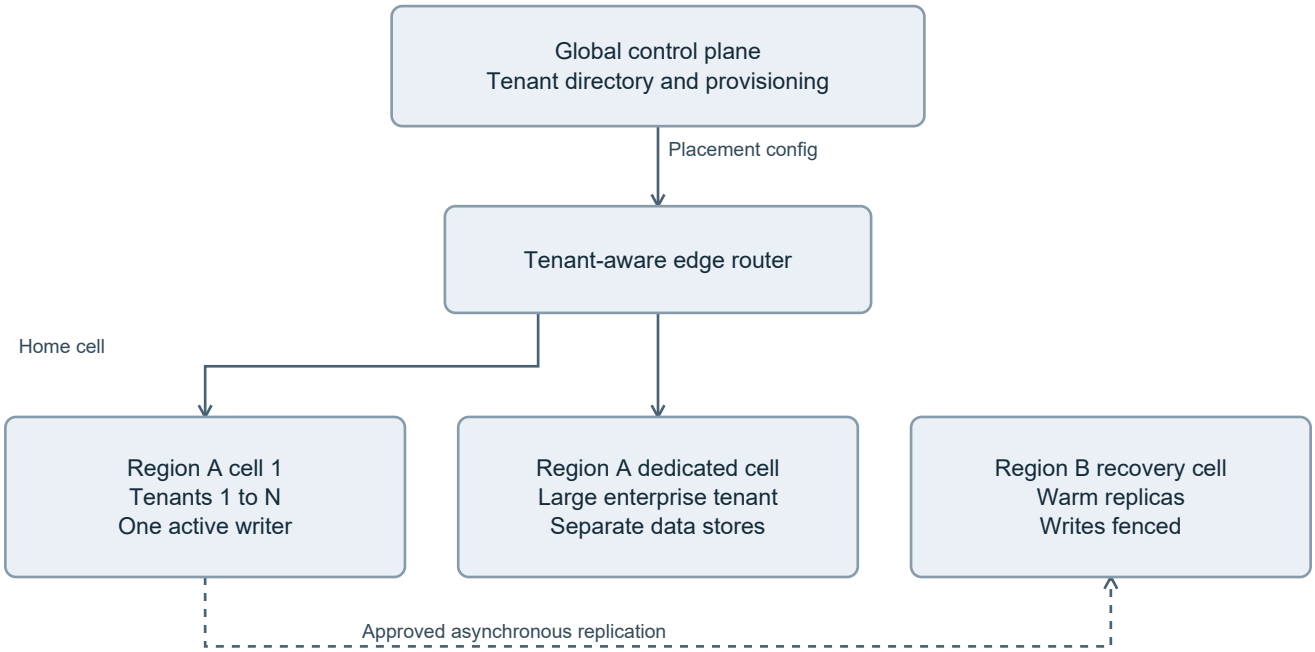


Figure 3. A tenant is active in one region at a time. Other tenants may be active in different regions; a global multi-master message database is not required.

Routing and availability

The directory maps tenant_id and provider phone_number_id to home_region, cell_id, and placement_epoch. Gateways cache signed snapshots. Validate account mapping after webhook signature verification. Use a region-specific webhook route where the provider configuration supports it; otherwise a minimal routing ingress must forward each item to its home region without storing message bodies in unauthorized locations.

Run at least three replicas for ingress and critical stateless services, spread across zones. Use anti-affinity, disruption budgets, readiness probes, and connection draining. Every cell has its own database cluster, event broker allocation, workflow namespace, quotas, and credentials boundary. Size the cell for the loss of one zone, not just average load.

Isolation and migration

Start with several bounded cells rather than one shared global database. Dedicated-cell placement is a provisioning option, not a fork of the software. Move a tenant by quiescing new writes, draining its work, copying data and dedupe history, checking counts and checksums, advancing the placement epoch, and resuming only in the target. Old workers must reject a newer epoch to prevent split-brain sends.

Select regions from customer residency contracts, provider availability, latency tests, and operational coverage. Cross-region backups, AI inference, telemetry, support access, and routing metadata all need the same placement policy. A tenant prohibited from cross-border replication needs an approved local recovery design with a different disaster profile.

5 Service boundaries and ownership

Service group	Owns	Key interface
Identity and tenant control	Organizations, users, roles, SSO settings, placement, entitlements	OIDC / SAML bridge, SCIM, signed placement snapshots
Channel account service	WABA and number mapping, credentials references, health, API version	Onboarding commands; channel.account.changed
Webhook ingress and normalization	Signature validation, durable raw envelope, provider event parsing	Provider POST; message.received / status.observed
Conversation and inbox	Messages, thread state, assignment, read cursors, session window	Reply and claim commands; conversation.updated
Outbound gateway	Send intents, attempts, rate admission, provider requests, ambiguity	message.send.requested; message.dispatch.updated
Contacts and policy	Contact identities, consent evidence, suppression, templates	authorizeSend; consent.changed
Campaign service	Audiences, immutable run snapshots, schedules, recipient outcomes	campaign.start; recipient.enqueue
Automation service	Draft graphs, immutable versions, execution history, timers	flow.publish; execution.signal
AI and knowledge	Document versions, retrieval, inference policy, evaluated prompts	Answer or handoff result; tool authorization request
Integration and media	Connector credentials, sync cursors, webhook delivery, scanned assets	connector.job; media.ready
Realtime and analytics	Authorized fan-out; read models and aggregate metrics	Tenant-scoped stream; analytics queries
Billing and audit	Usage ledger, entitlements projection, administrative evidence	usage.recorded; audit.append

These are ownership boundaries; deploy them as independently scalable groups, not a service for every table. Each group owns its schema and migrations. Other groups call an API or consume a versioned event rather than writing its tables. A cell may share a physical PostgreSQL cluster initially, but it must enforce roles, connection budgets, and schema ownership. The outbound gateway remains the only service permitted to call the provider message-send endpoint.

6 Tenant onboarding and channel connection

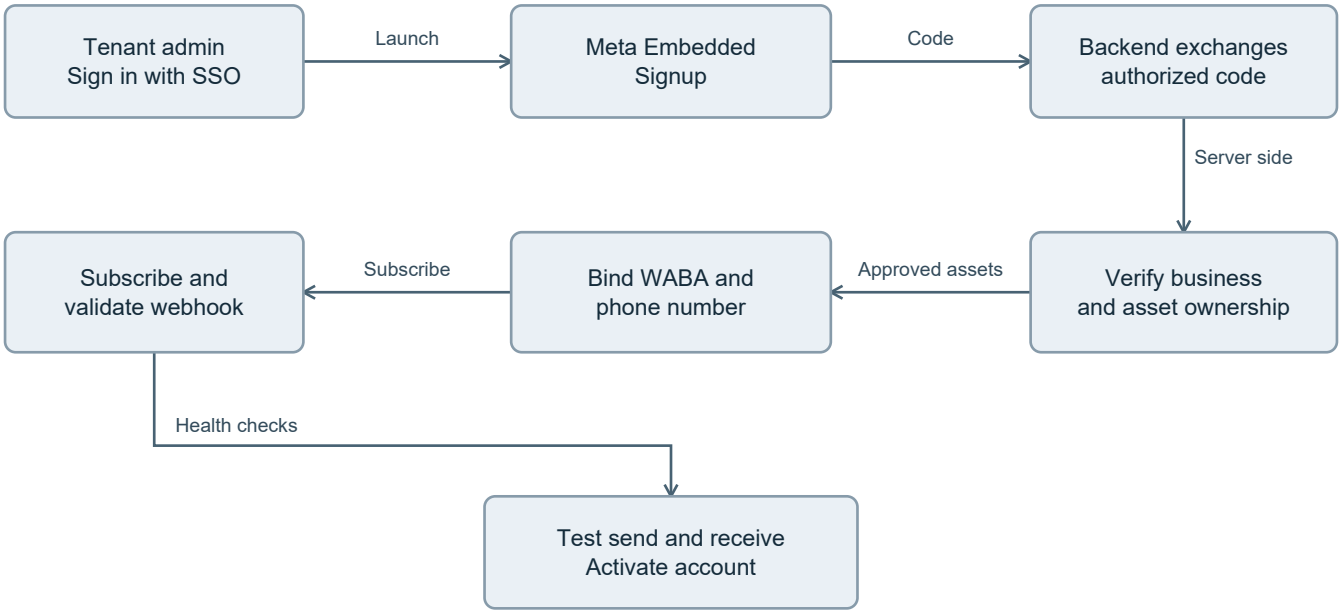


Figure 4. Activation is a resumable workflow. Every stage has a stored result, retry policy, and recoverable error state.

• Create the organization, home cell, administrators, retention policy, and entitlement record before connecting a channel. Bind signup to a server-generated nonce and authenticated tenant session; validate callback origins and expiry.

• Use Meta's supported Embedded Signup path for the approved partner model. Exchange the authorization code on the server and independently verify the granted assets. Do not trust a browser-supplied tenant or phone number mapping. [3, 4]

• Store WABA ID, phone number ID, business ID, credential reference, approved permissions, token health, and selected API version. Keep credentials encrypted in the secrets service. Permission scopes and token lifecycle depend on the integration path.

• Register or connect the number as required, subscribe the application to the appropriate account events, configure the verification endpoint, and perform real inbound and outbound checks. Activation requires both directions to work.

• Represent account state explicitly: pending_authorization, pending_verification, configuring, testing, active, degraded, revoked, disconnected. Revocation pauses sends and raises an actionable alert; it does not silently erase history.

Coexistence with the WhatsApp Business app and account migration need dedicated onboarding branches and eligibility checks. Do not assume every number supports them. Complete Meta application review, required access approvals, business verification, privacy/deletion routes, and commercial onboarding before general customer signup. [3, 4]

7 Inbound webhook pipeline

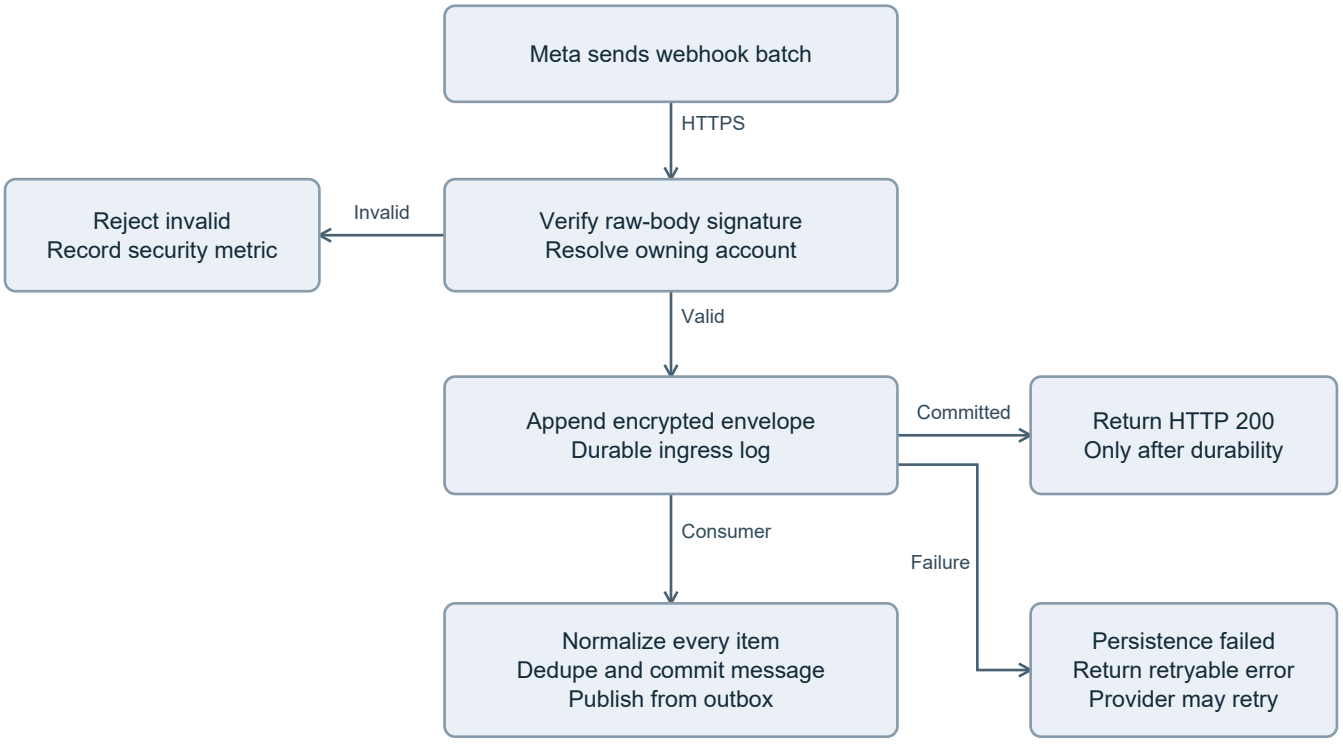


Figure 5. Fast acknowledgment means durable acceptance, not completed bot processing. The provider is never kept waiting for AI, CRM, or media downloads.

Validate HMAC-SHA256 over the original request bytes using the correct application secret and constant-time comparison. The GET subscription challenge is a separate setup check. Limit payload sizes, validate content type, and safely parse all entries and changes in a batch. Preserve authenticated unknown event types for bounded investigation instead of breaking the whole batch. [6]

Append a compact encrypted envelope to replicated Kafka with acknowledgments from the configured in-sync replica set. For a request spanning partitions or tenants, acknowledge only when every required append succeeds; partial retries are handled by deduplication. If storage is unavailable, return an appropriate retryable failure and alert. Never acknowledge first and place the event only in memory.

A normalizer maps provider identifiers to a canonical event. The conversation worker inserts the message, updates last_customer_message_at using the maximum valid provider timestamp, increments a conversation sequence, and writes an outbox row in one transaction. Uniqueness on tenant, channel account, and provider message ID rejects duplicate inbound delivery. Commit consumer progress only after durable work; retries may occur.

Status events use a separate key including provider message ID, status, and provider timestamp or stable payload hash. Archive the raw envelope by retention policy, with a pointer for diagnostics. Realtime, automation, analytics, and integrations consume independent downstream subscriptions.

8 Outbound message pipeline

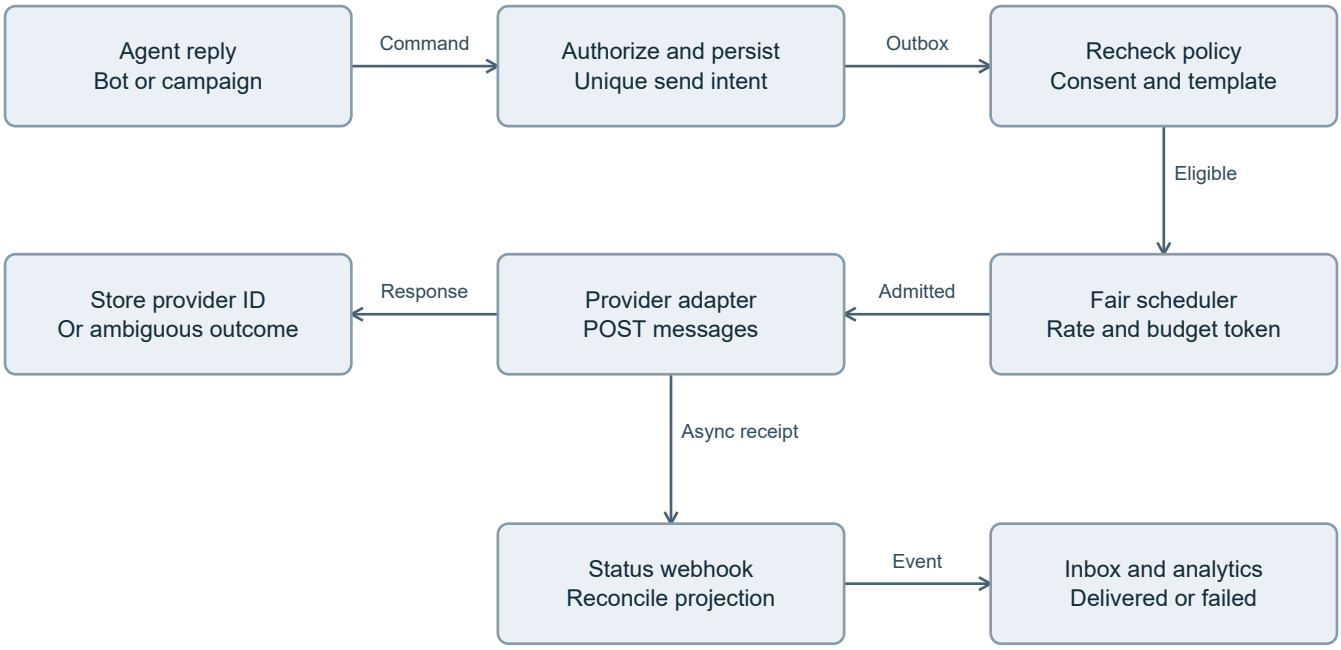


Figure 6. Provider acceptance and device delivery are different milestones. Every source uses the same outbound gateway and policy checks.

Accept a caller idempotency key scoped to tenant and operation. Persist the request hash and result; the same key with different content returns a conflict. Commit the send intent and its outbox record before returning HTTP 202 with a stable internal ID. The UI displays queued immediately and later follows observed provider status. [5]

Before dispatch, re-read suppression, account health, template status, customer-service-window eligibility, ownership generation, message expiry, and remaining budget. Check both tenant and sender limits. A delayed campaign item may no longer be eligible when it reaches the front of the queue. Policy failures are recorded as blocked, not endlessly retried.

Persist an attempt and ownership fence before the network call. On success, attach the provider message ID. On an unambiguous permanent error, fail with a normalized reason. Retry a documented retryable rejection with jitter and Retry-After handling. A timeout after request transmission is unknown: the provider may have accepted it. Do not blindly resend.

Where supported, attach an opaque internal correlation value to provider callbacks; confirm support for the selected API version. Otherwise correlate with available status evidence and quarantine unresolved attempts for operator review. Internal deduplication cannot guarantee exactly-once external delivery without a provider-supported idempotency contract. Store that limitation in the operational runbook.

9 Events ordering and reliable state changes

Canonical event envelope

```
event_id: UUID
event_type: message.received.v1
tenant_id: t_123    cell_id: eu_02
aggregate_id: conv_789    aggregate_seq: 104
channel_account_id: ca_456
provider_message_id: opaque_provider_id
occurred_at: provider timestamp    received_at: platform timestamp
trace_id: trace_abc    causation_id: event_or_command_id
schema_version: 1    data_ref: encrypted_payload_reference
```

Use a schema registry, additive changes by default, and versioned breaking events. Keep secrets and large media out of the broker. Events identify the tenant, but consumers must also validate that their credentials and storage scope permit that tenant. Sign or authenticate all producer connections; a tenant_id field is not authorization.

Consistency rules

• Partition message and conversation events by tenant_id plus conversation_id. This keeps related events on one ordered stream, while many conversations run in parallel. Kafka ordering is within a partition, not across all topics or producers. [8]

• Allocate aggregate_seq under a row lock or optimistic version check in the owning service. Relay each aggregate’s committed outbox rows in sequence. Consumers detect gaps and reconcile; provider arrival order remains independent of business event time.

• Use a transactional outbox for database state plus event publication. The publisher can crash after publishing and before marking completion, so consumers need a durable processed-event record and idempotent effects. Broker acknowledgments alone do not make a database transaction atomic. [7]

• Commit a consumer’s processed-event marker with its business write. For remote effects, commit a stable action intent first and use the remote provider’s idempotency mechanism if available. Dedupe records must survive the full replay and retry horizon.

• Retry transient failures with exponential backoff and jitter; quarantine malformed or exhausted items with reason, original event ID, and diagnostic pointer. Replays preserve event and action identities. Never replay historical events into live sends without an explicitly authorized replay mode.

Separate ingress, conversation, outbound interactive, outbound bulk, status, integration, and analytics workloads. Use tenant quotas and weighted scheduling so a large campaign cannot consume the worker and provider capacity reserved for live replies. Poison messages block or isolate the affected conversation rather than silently violating its ordering.

10 Core data model

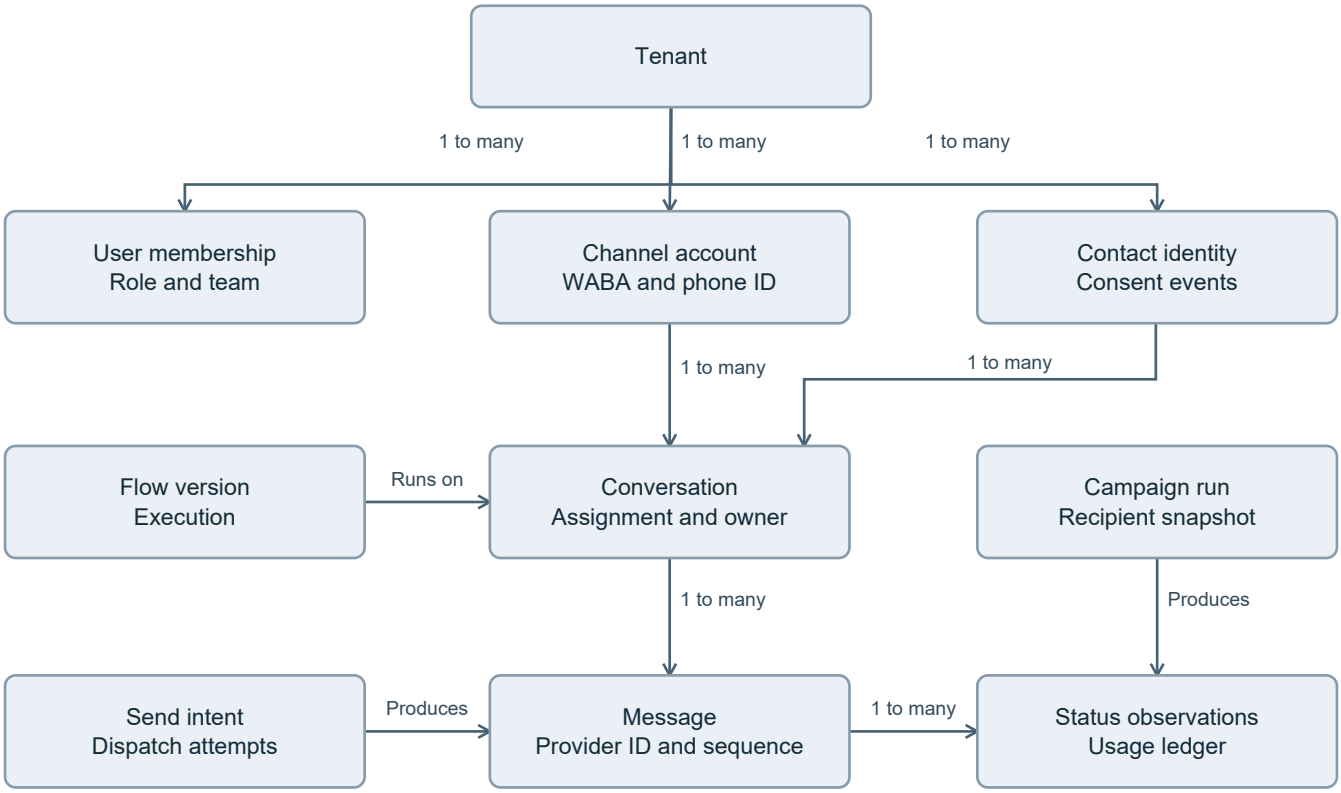


Figure 7. Conceptual relationships; every tenant-owned relation carries tenant_id. Send intents and attempts are separate from observed provider messages.

A contact can have multiple channel identities, and a person may be represented differently across channels. Normalize phone numbers to E.164 where applicable, store provider identifiers separately, and use explicit verified merges. Do not automatically merge tenants or identities solely because display names match.

A conversation belongs to a tenant, channel account, and contact identity. Define an explicit policy for reopening versus creating a new thread, while tracking the provider's customer-service window separately from the ticket lifecycle. A closed support ticket does not mean the messaging window is closed.

Keep message bodies and attachments separate from audit evidence and analytics facts. Messages contain direction, type, sequence, timestamps, references, and retention metadata. Append provider status observations and derive the current display state without discarding late evidence. A read receipt need not be preceded by every lower status in arrival order.

Index high-volume access paths by tenant and conversation with a monotonic sequence or timestamp plus stable tie-breaker. Use cursor pagination; avoid deep OFFSET scans. Apply tenant-prefixed composite foreign keys so a valid message ID cannot accidentally reference another tenant's conversation.

11 Storage constraints and retention

Record	Required constraint or index	Retention proposal
Channel account	Unique provider + phone_number_id within the platform mapping; explicit ownership transfer	Active life plus approved deprovision window
Message	Unique tenant + account + provider_message_id; index tenant + conversation + seq	90 days hot by default; customer-configurable archive
Send intent	Unique tenant + operation + idempotency_key; request hash	At least retry and replay horizon; linked to retained messages
Dispatch attempt	Intent + attempt number; fenced claim generation	30 to 90 days operational history, then reduced evidence
Consumer inbox and outbox	Unique consumer + event_id; pending publication index	At least maximum replay horizon plus safety margin
Campaign recipient	Unique run_id + contact_identity_id; due_at index	Campaign reporting term; purge raw audience data per policy
Consent event	Append-only evidence, source, purpose, channel, observed_at	Contractual retention; current suppression retained as needed
Flow version and execution	Immutable version hash; unique execution step action key	Version retained while executions or audit need it
Media and knowledge objects	Tenant key prefix, checksum, MIME, malware state	Typically 30 to 90 days; legal hold overrides expiry
Audit and usage ledger	Append-only event ID; reconciliation keys	Term approved by security, finance, and customer contract

Physical placement

Use managed PostgreSQL per cell for transactional records, with a writer and zone-separated failover capacity. Partition large message and event tables by time and, where justified by measurements, tenant hash. Maintain a separate unpartitioned or appropriately keyed dedupe registry when a global uniqueness requirement crosses partitions. Read replicas serve historical reporting only when replica lag is acceptable.

Use Redis for ephemeral presence, cache, leases with fences, and quota counters. Use object storage for encrypted media, raw event archives, exports, and backups. Add a search index for full-text inbox queries and a separate columnar analytics store when transaction-safe reporting becomes too expensive. Derived stores are rebuildable and cannot authorize a send.

Deletion is a tracked workflow across PostgreSQL, search, object versions, knowledge chunks, analytics identifiers, and connector caches. Expire backups by policy and reapply deletion tombstones after a restore. Isolate minimal immutable audit evidence from deletable message content; avoid putting plaintext conversations in permanent logs. The durations above are proposals, not legal retention requirements.

12 Consent templates and send policy

One policy decision at every send boundary

The policy service returns allow, block, or defer with a reason code, policy version, and decision timestamp. Inputs include tenant, channel account, contact, message purpose and type, last inbound customer timestamp, template state, suppression evidence, account restrictions, schedule, and budget. Record the decision on the send intent. Recheck time-sensitive facts immediately before the network call.

Condition	Platform action
No valid marketing permission or contact opted out	Block promotional sending; preserve evidence and update suppression immediately.
Open customer service window	Permit supported response types subject to current channel rules and the user’s purpose.
Outside customer service window	Require an eligible approved template under the current provider policy; block free-form fallback.
Template paused, rejected, deleted, or wrong language	Block affected sends and mark campaign recipients with a specific reason.
Account restricted, revoked, or provider rate exceeded	Pause or defer according to error class; surface the cause to the tenant.
Quiet hours, expired campaign, frequency cap, budget limit	Defer until eligible or permanently expire; never send stale promotions.

Consent and suppression pipeline

Store permission as evidence: identity, purpose, channel, source, time, policy text/version, and revocation. A contact imported from a CSV or CRM is not automatically eligible for marketing. Match opt-out signals using configured phrases and channel controls; apply suppression transactionally before generating further automated replies or campaign work. Cancel pending promotional intents and explain any already-in-flight race in the audit record.

Template lifecycle and pricing

Store provider template ID, name, language, category, variable schema, approval status, revision, and last synchronization time. Validate sample variables at creation, synchronize status changes, and revalidate again at dispatch. A local published template is not evidence of provider approval.

Meta’s public pricing page describes delivered-message pricing and a 24-hour customer-service window. Pricing and eligibility are different rule sets: an eligible send is not necessarily free. Use effective-dated rate cards and policy configuration rather than hard-coded rates or indefinite assumptions about free message categories. [9]

13 Campaign and sequence pipeline

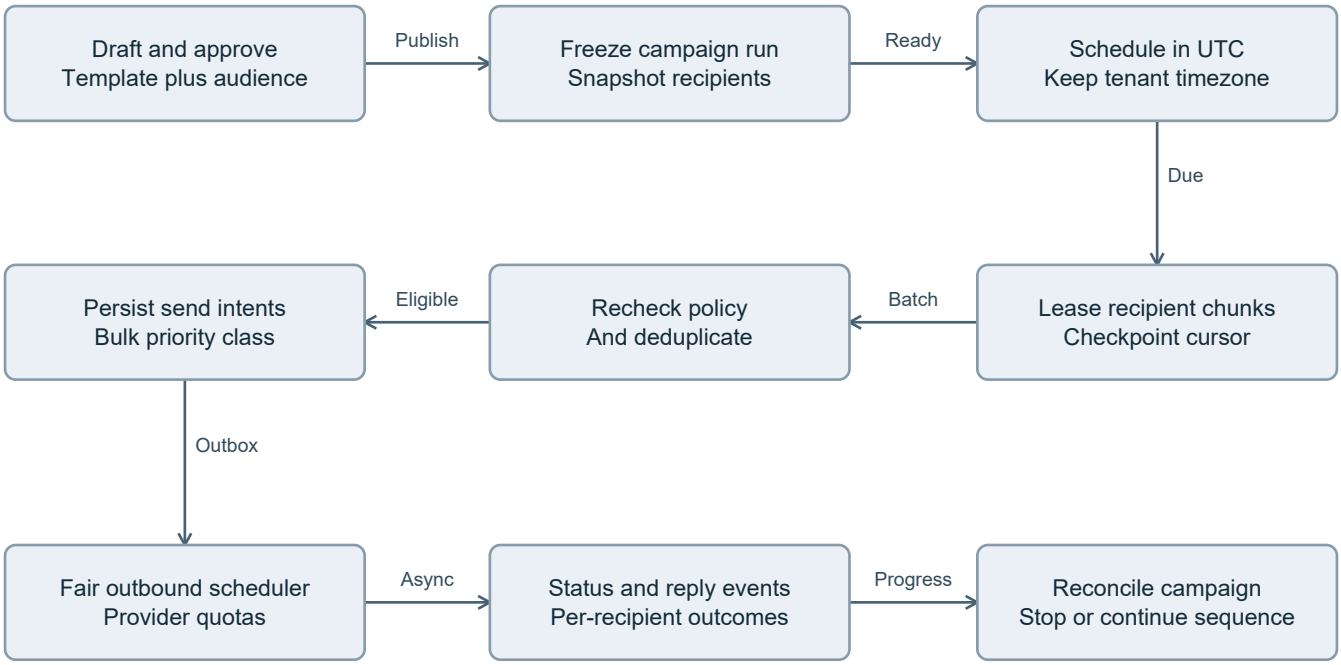


Figure 8. The audience snapshot makes the run reproducible, while a live policy check protects against later opt-outs or template changes.

Store filter definition, materialized recipient IDs, template version, variables, initiating user, approval evidence, and maximum spend. Resolve recipients in chunks so a million-contact audience does not become a single database transaction. A unique run/recipient key prevents a restarted dispatcher from enqueuing the same logical message twice.

Use weighted fair scheduling across tenants, with separate interactive and bulk queues. Apply sender throughput, recipient-pair limits where applicable, tenant budget, campaign concurrency, and account-quality constraints. Limits come from supported provider entitlements and current configuration, not a universal advertised messages-per-second value. Stop producers when backlog age or broker capacity exceeds limits.

Pause prevents new admissions and cancels eligible queued intents; it cannot recall requests already accepted by the provider. Keep pending, blocked, submitted, delivered, read, failed, unknown, and cancelled counts separate. Completion means all recipients have a terminal local outcome or a recorded unresolved condition, not that all recipients read the message.

For drip sequences, schedule durable timers per enrollment and stop on reply, conversion, opt-out, or agent takeover. Recheck eligibility after every wait. Store the tenant’s timezone and daylight-saving behavior for recurring schedules; convert each next occurrence to UTC instead of adding a fixed 24 hours.

14 Visual chatbot builder and publication

The graph is a versioned program

Build the editor in React and TypeScript with a graph component such as React Flow. The canvas stores a typed JSON graph, not executable JavaScript. Each node has an ID, type, input schema, output schema, validation rules, error route, and configuration. Edges refer to named output ports. A server-side compiler validates and publishes an immutable graph version with a content hash.

Node type	Execution meaning	Required controls
Trigger	Start from inbound text, webhook, schedule, or explicit enrollment	Priority, exclusivity, tenant scope and duplicate trigger key
Send message	Create a standard outbound intent	Template eligibility, variable typing and action idempotency
Collect input	Wait for and validate the next matching user response	Timeout, retry prompt, cancellation and sensitive-field handling
Condition and switch	Evaluate typed expressions against execution variables	No arbitrary code; bounded evaluation; explicit default branch
Wait and schedule	Resume after a durable timer or an external event	Expiry, timezone semantics, cancellation and generation check
HTTP or connector	Invoke a registered integration action	Destination policy, schema, scoped secret, timeout and retry class
AI answer	Retrieve knowledge, generate, validate, then create reply intent	Tenant filter, grounding, tool allowlist, budget and fallback
Handoff and end	Transfer ownership to an agent or finish execution	Cancel bot sends; persist reason and next owner

Compile and release

Reject missing start nodes, unreachable required steps, dangling edges, invalid variables, unsupported channel features, and loops without a bounded counter or wait. Enforce maximum graph size, transitions per event, lifetime, outbound messages, and parallel branches. Keep secrets as references to approved connectors. Simulate the graph with canned events and clock control before release.

Draft → validated → approved → published → retired is the publication lifecycle. An execution stays pinned to its starting version. A new publish changes new executions only; migration of active executions is a separately tested operation. Record who changed and approved the flow. A canary release can route a small tenant-selected share of new sessions to a version with an immediate rollback switch.

The platform’s automation graph is distinct from WhatsApp Flows, Meta’s in-chat structured forms. Add WhatsApp Flows as a channel feature with a separate definition, publication, endpoint-security, and response adapter. Do not treat a canvas graph as a provider-native form. [3]

15 Durable automation runtime

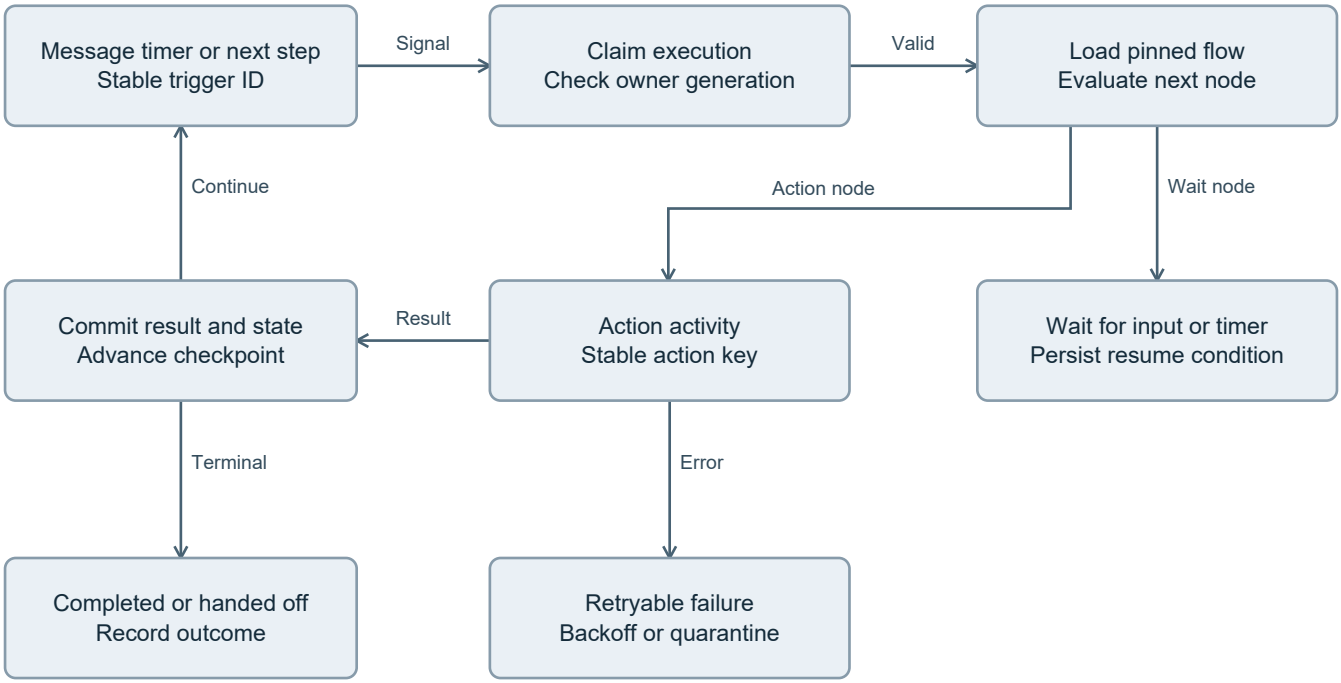


Figure 9. State survives restarts. A stored wait resumes through a new event. Activity results are recorded and retries reuse the same action identity.

Use a durable workflow engine such as Temporal for long-lived conversations, waits, and integration orchestration. Run its frontend, history, matching, and worker services with managed persistence across zones, or select a managed deployment that meets the tenant's residency requirements. Kafka distributes events; the workflow engine owns execution history and timers. They serve different purposes. [10]

Use a stable workflow ID derived from tenant and execution ID. The signaling adapter records delivery progress so a crash after signaling can retry safely; the workflow deduplicates incoming event IDs. Continue or segment long histories before their configured size limit. Encrypt sensitive payloads and store large bodies by reference.

Workflow decisions must be deterministic on replay. Network calls, wall-clock-dependent business lookups, AI inference, and random IDs belong in recorded activities or supported workflow primitives. Activities may retry, so every side effect uses an action key such as tenant/execution/node/iteration. A database uniqueness check prevents duplicate intent creation. [10]

Human takeover advances the conversation owner generation. The runtime stops or parks the flow; outbound dispatch checks the same generation immediately before sending. Already-transmitted calls cannot be recalled. Timer events carry execution generation, making a stale reminder harmless after cancellation or a new session. Unhandled input follows a declared fallback rather than starting overlapping bots.

16 Shared inbox and realtime collaboration

Read and write paths

The browser first loads an authorized conversation snapshot from the inbox API and receives its sequence cursor. It then subscribes through the realtime gateway. Each streamed event carries conversation ID and sequence; the client detects missing sequences, reconnects with its last cursor, and fetches a delta or full snapshot. Realtime fan-out may use ephemeral pub/sub because the durable database remains the recovery source.

User action	Server behavior	Concurrency rule
Open inbox	Return paginated conversations within role and team scope	Server filters every query; UI filtering is not security.
Claim conversation	Compare expected assignment version and set owner	One successful claim; stale callers receive conflict and new state.
Type a reply	Publish expiring presence after subscription authorization	Presence is advisory; it never grants ownership.
Send reply	Persist message intent with client-generated idempotency key	Check current owner/version; queued status until provider evidence.
Take over from bot	Advance ownership generation and cancel pending automation	Recheck generation at dispatch; log in-flight exceptions.
Mark read or close	Update the agent read cursor or ticket state	Do not overwrite another agent’s cursor or provider receipt.
Reconnect	Resume from last durable sequence or load snapshot	Deduplicate by event ID; bounded buffers and backpressure.

Agent routing

Route by team, language, skills, working hours, availability, and explicit capacity. Assignment service claims a conversation using optimistic concurrency and records the decision. A round-robin counter alone is insufficient when agents disconnect or reach their workload limit. Requeue abandoned assignments after a configured lease and preserve customer priority and SLA age.

A per-conversation ownership state machine is BOT_ACTIVE → HANDOFF_PENDING → AGENT_ACTIVE → RESOLVED, with explicit transitions back to automation. Treat internal notes, draft replies, and customer-visible messages as separate object types and permissions. Restrict transcript export and phone-number visibility independently of conversation access.

Realtime infrastructure

Spread gateways across zones, cap subscriptions per connection, heartbeat idle sessions, and drain connections on deploy. Do not depend on sticky sessions for correctness. Disconnect slow clients with a resume cursor instead of allowing unlimited memory growth. Revalidate authorization on subscribe and permission changes; revoke existing streams when team membership or tenant access is removed.

17 AI knowledge and response pipeline

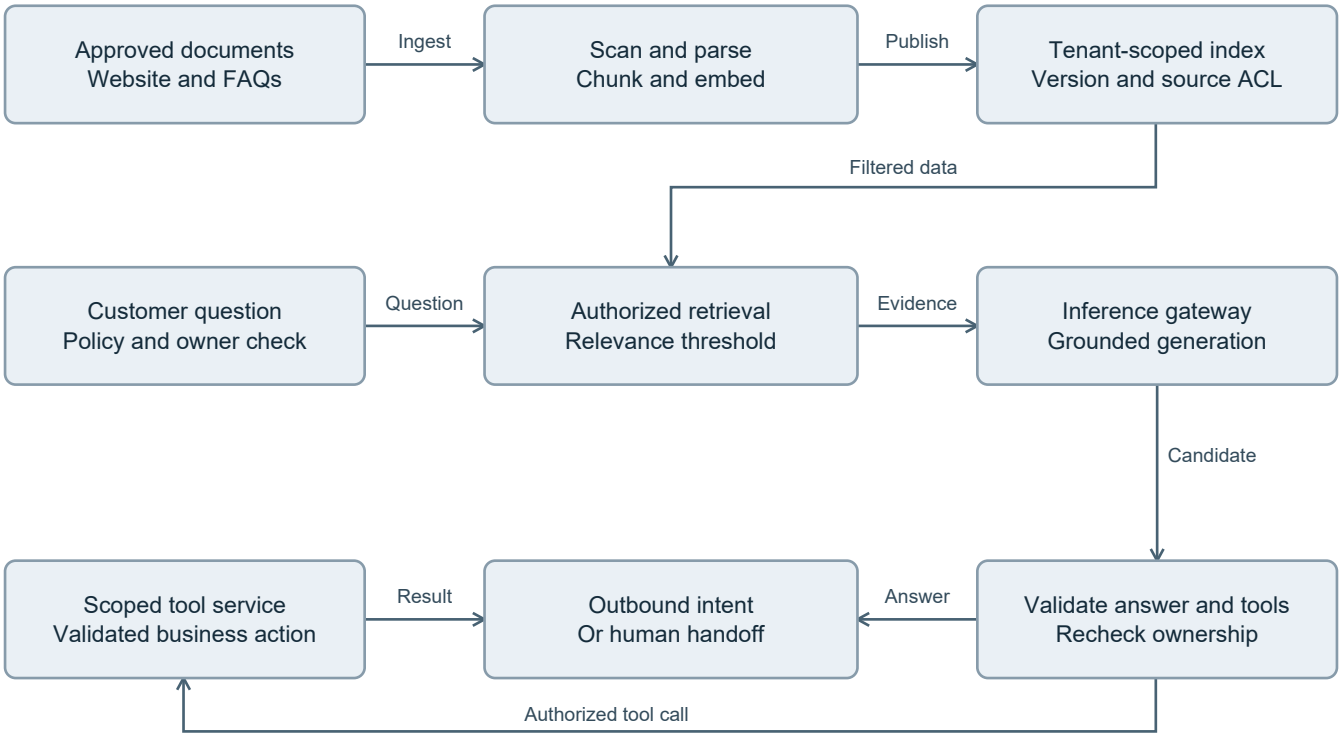


Figure 10. Documents and model output are untrusted inputs. All data access and business actions pass deterministic authorization outside the model.

Knowledge ingestion validates file types, scans malware, parses in a sandbox, strips active content, chunks text, and attaches tenant, document version, source, access tags, and checksum. Website ingestion uses a restricted fetcher with crawl limits and SSRF controls. Build a new index version and switch atomically; deleted sources invalidate their chunks and cached answers. [12]

At inference, identify the tenant and conversation from authenticated context, select only authorized sources, and retrieve bounded evidence. Pass a short relevant conversation summary and explicit business policy to the model. Require source-linked internal evidence for factual replies; use a tested relevance threshold and abstention route rather than treating the model's self-reported confidence as reliable.

The model proposes an answer or a typed tool call. A separate service validates tool name, argument schema, tenant ownership, user entitlement, and action policy. Read-only order lookup and a refund are different permissions. Sensitive actions require the business's configured confirmation or approval. External tools use scoped credentials and idempotency keys; retrieved text cannot grant new capabilities.

Define latency and token budgets, model/provider routing, data-region constraints, redaction, bounded retries, and fallback to an agent. Keep AI out of the webhook acknowledgment path. Record model version, prompt version, retrieved document IDs, tool decisions, latency, and usage with retention controls. Evaluate hallucination, tenant leakage, injection resistance, task completion, and handoff quality before auto-send.

18 Integration media and commerce pipelines

Connectors and customer webhooks

Implement a connector contract: authorize, refresh_credentials, validate_config, execute_action, pull_changes, normalize_event, and health. Each installation has tenant ownership, credential reference, scopes, object mapping, cursor, and sync direction. Use provider webhook notifications when available and scheduled cursor-based reconciliation to recover missed updates. Store source version and provenance to avoid endless CRM-to-platform-to-CRM update loops.

Inbound customer webhooks validate the connector’s signature, persist an event, and acknowledge quickly. Outbound customer webhooks include event ID, timestamp, schema version, and an HMAC signature over a documented canonical payload. Retry with backoff and a bounded horizon, then expose a delivery log and replay action. Receivers must deduplicate; your platform cannot promise exactly-once delivery to a customer endpoint.

Protect HTTP nodes, website crawlers, media fetchers, and webhook delivery against SSRF. Allow only approved schemes and destinations; reject private, loopback, link-local, metadata, and other non-public addresses for both IPv4 and IPv6. Resolve and validate at connection time, control redirects, and revalidate each redirect. Use an egress proxy to resist DNS rebinding and prevent access to private service networks. [12]

Media transfer

Inbound media metadata creates a download job. A restricted worker retrieves the provider object with a scoped token, enforces byte limits, sniffs MIME type, scans and optionally transcodes in isolation, encrypts the result in object storage, and emits media.ready. The inbox shows a pending placeholder until scanning passes. Quarantine failed assets without blocking unrelated text messages.

For outbound uploads, issue a short-lived signed upload permission for a random tenant-scoped key with a size cap. Finalization verifies ownership, checksum, file type, and scan state. The provider adapter uploads or references the approved media using the channel’s supported mechanism. Authorized downloads use short-lived links; do not log signed URLs or make conversation attachments publicly readable.

Catalogs and transactional systems

Treat CRM, order management, stock, and payment systems as the source of truth for their domains. Cache catalog displays with a version and freshness limit. Confirm price and availability against the transactional system before creating an order. Never mark an order paid from a chat message; use a verified payment-provider event and reconcile independently. Payment details should remain on the payment provider’s hosted collection surface.

WhatsApp Flows with data exchange need a separate endpoint, tenant-bound session token, replay/expiry checks, and the provider’s required encryption implementation. The underlying booking or order action still passes through the same authorized, idempotent connector service. [3]

19 Security and tenant isolation

Identity and access

Use an established identity provider for OIDC login and enterprise SAML federation; support SCIM provisioning where required. Require MFA for administrators and privileged operators. Maintain tenant memberships separately from global identity, and issue short-lived sessions or tokens with audience and scope. Recheck server-side roles for every operation and audit role changes, exports, credential changes, and flow publication.

Define owner, administrator, supervisor, agent, analyst, developer, and billing roles, with team and conversation-level restrictions. Service-to-service calls authenticate workload identity and propagate a validated tenant context. The API gateway strips externally supplied internal identity headers before adding trusted ones. Customer API credentials are scoped, hashed at rest where possible, rotatable, and never equivalent to operator credentials.

Database and storage controls

Enforce tenant_id in queries and composite keys, backed by PostgreSQL row-level security on tenant tables. Use non-owner application roles without BYPASSRLS; FORCE ROW LEVEL SECURITY where appropriate. Set tenant context locally inside a transaction and reset pooled connections. Admin and migration roles stay separate. Test cross-tenant reads, writes, joins, exports, background jobs, and error messages. RLS is defense in depth, not a substitute for application authorization. [13]

Scope object storage paths, signed URLs, search filters, vector retrieval, caches, broker ACLs, and realtime subscriptions. A missing tenant filter in a search or AI index can bypass an otherwise secure database. Use dedicated credentials and encryption keys for higher-isolation tenants. Encrypt disks, objects, backups, and sensitive workflow payloads; keep secrets in a managed vault with rotation and audited access.

Privileged access and application defenses

Give support staff metadata-only access by default. Require a time-limited, auditable elevation for customer content, with tenant approval where the contract requires it. Keep a separate admin domain and private operator path. Use just-in-time access, session recording where supported, and emergency access reviewed after use.

Escape rendered messages and notes, sanitize HTML, enforce a strict content-security policy, protect state-changing cookie requests against CSRF, validate uploads, and restrict URL fetchers. Rate-limit login, exports, API creation, media, and resource-intensive queries. Keep cloud metadata unreachable from untrusted execution. Isolate connector and document-processing workloads from core credentials.

Run dependency and image scanning, secret detection, SBOM generation, signed-image verification, and periodic penetration tests. Certifications such as SOC 2 or ISO 27001 are organizational programs with evidence requirements; this architecture does not confer certification. Map controls to actual customer contracts and applicable law before launch.

20 Public API and event contracts

Send a message

```
POST /v1/channel-accounts/ca_456/messages
Authorization: Bearer <scoped-platform-token>
Idempotency-Key: order-987-dispatched-v1

{
  "contact_id": "ct_123",
  "type": "template",
  "template": {"id": "tpl_22", "language": "en",
    "parameters": {"order_number": "987"}},
  "client_reference": "order_987",
  "expires_at": "2026-09-30T12:00:00Z"
}

202 Accepted
{"message_intent_id":"mi_001","state":"queued",
 "status_url":"/v1/message-intents/mi_001"}
```

Infer tenant from authenticated credentials and verify that ca_456 and ct_123 belong to it. The request contains an internal template ID resolved to a provider template by the channel adapter. This is a proposed platform contract, not a literal Meta request body. A successful response confirms durable intent acceptance, not provider delivery.

Endpoint	Behavior
GET /v1/conversations?cursor=...	Stable cursor pagination with team authorization; returns next cursor and snapshot sequence.
POST /v1/conversations/{id}/claim	Requires expected_version; returns 409 when ownership changed.
POST /v1/flows/{id}/publish	Validates graph, checks approval, creates immutable flow_version_id.
POST /v1/campaign-runs	Validates audience, template, schedule and budget; returns asynchronous run ID.
POST /v1/webhook-subscriptions	Creates a verified, signed-event destination with SSRF controls.
GET /v1/message-intents/{id}	Returns local state, provider ID if known, observed receipts and safe error details.

Use a consistent error shape: code, human-readable message, retryable flag, request_id, and field errors. Return 401 for invalid authentication, 403 for forbidden actions, 409 for conflicts, 422 for invalid payloads, and 429 with retry information for quota limits. Avoid leaking whether another tenant’s resource exists. Document idempotency retention, pagination, webhook retries, versioning, and deprecation windows.

Enforce schema and size limits at the edge and application layers. Version OpenAPI and event schemas in source control, run consumer contract tests, and provide a sandbox with synthetic contacts and provider stubs. Pin the Meta Graph API version per adapter deployment and test upgrades before changing live tenant traffic.

21 Analytics billing and audit pipelines

Analytics

Publish canonical events from each owning service to a regional analytics pipeline. Deduplicate by event ID, apply event-time windows with a defined lateness allowance, and write dimensioned facts to a columnar store. Separate operational inbox reads from large analytic scans. Tenant-facing dashboards must disclose freshness and distinguish provider-accepted, delivered, read, replied, and converted events.

Define metrics before implementation: delivery rate uses delivered divided by provider-accepted sends with a declared observation window; reply rate counts unique replying recipients; first-response time measures first qualifying business response after the customer’s initiating event. Exclude internal notes and document whether bot responses count. Missing read receipts are unknown, not proof of unread messages.

Metering and financial reconciliation

Create an append-only usage ledger for platform seats, accepted billable platform actions, AI tokens, storage, and any provider charges passed through. Each record contains tenant, source event, meter, quantity, unit, effective rate-card version, timestamp, and reconciliation status. Uniqueness prevents duplicate callbacks or event replay from duplicating charges.

Prefer direct Meta billing to the tenant when the commercial model permits it. If reselling provider spend, maintain reservations, final settlement, credit notes, taxes, currency treatment, and invoice reconciliation as a separate financial domain. Never charge every send attempt as if it were delivered. Preserve estimated, observed, and invoiced cost separately and reconcile to provider evidence. [9]

The send gateway reads a local entitlement and budget projection so a billing dashboard outage does not stop live support. Expiring cache and conservative limits prevent uncontrolled spend. Campaign launch reserves a configured maximum budget; recipient settlement releases unused reservations. A subscription webhook is deduplicated and verified before entitlement changes.

Audit evidence

Record actor identity, tenant, action, target, timestamp, request ID, source IP or device context when justified, previous and new configuration hashes, and approval references. Store security-critical evidence in an immutable log account with restricted writers and retention controls. Keep sensitive message bodies out of audit entries whenever IDs and hashes are enough.

Analytics rebuilds must not trigger sends or rebill usage. Use separate consumer identities, read-only processing modes, and destination topics. Reconciliation jobs compare source message counts, intent outcomes, provider observations, ledger totals, and campaign recipient totals; discrepancies create a review task rather than an automatic customer charge.

22 Availability targets and observability

The following are proposed internal targets under the capacity envelope on page 25. Measure them before committing contractual SLAs. Provider outages, device connectivity, template review, and AI latency must be visible separately from platform availability.

Indicator	Initial target	Measurement boundary
Webhook durable acceptance	99.95% availability; p99 below 500 ms	Valid webhook arriving at platform edge to durable log acknowledgment; provider delivery delay excluded.
Core API	99.95% availability; p95 reads below 300 ms	Authenticated valid requests at platform edge; complex exports measured separately.
Message visible in inbox	p95 below 2 seconds	Durable inbound acceptance to authorized browser event in a connected session.
Interactive send admission	p95 below 2 seconds	Persisted eligible intent to start of provider request; exhausted provider quota reported separately.
Simple bot reply intent	p95 below 3 seconds	Inbound durable acceptance to persisted reply intent; external AI/CRM measured separately.
AI answer	p95 below 12 seconds; bounded timeout	Eligible AI request to answer or handoff; selected model and evidence size recorded.
Same-region zone failure	No loss of acknowledged committed records within configured durability	Verified database/broker failover test; maintain quorum and spare capacity.
Regional recovery	RPO at most 5 minutes; RTO at most 60 minutes	Target for approved cross-region replicas; replication lag and promotion drill determine achievable result.

A 99.95% monthly availability objective allows about 21.6 minutes of unavailability in a 30-day month. Use error-budget burn alerts over short and long windows. Separate platform failure from tenant misconfiguration in reporting without hiding real customer impact.

Signals and alerts

Propagate trace_id, tenant_id, conversation_id, intent_id, and provider message ID through logs and traces. Redact tokens, message bodies, and signed URLs. Use bounded metric labels rather than one time series per message or customer. Sample traces while retaining failure exemplars with access controls.

Alert on ingress acknowledgment errors, oldest unpublished outbox age, consumer lag, oldest eligible interactive intent, ambiguous sends, provider 429/5xx rates, token revocation, DLQ growth, database saturation, replication lag, and cost anomalies. Give every alert an owner, customer-impact description, dashboard, and tested runbook. Run synthetic inbound/outbound checks using controlled accounts.

23 Capacity model and cost drivers

Planning example: 10,000 tenants, 20,000 connected agents, and 20 million message records per day across 20 cells. Assume 60% outbound messages and a 10-times daily-average burst. This is an engineering sizing scenario, not an expected launch volume or a measured benchmark.

Calculation	Result and implication
Message average = 20,000,000 / 86,400	231.5 messages/s globally; approximately 11.6/s per cell at even distribution.
Message burst = average × 10	2,315 messages/s globally; 116/s per cell before hot-tenant skew.
Events = 20M messages + 3 × 12M outbound status events	56M events/day; 648/s average; 6,481/s at 10-times burst. Verify actual status multiplicity.
Event bytes = 56M × 2 KB	112 GB/day uncompressed. Seven-day log ≈ 784 GB logical; three replicas ≈ 2.35 TB before overhead.
Message rows = 20M × 1.5 KB	30 GB/day logical; 90 days ≈ 2.7 TB before indexes, replicas, WAL and backups.
Media = 5% of messages × 250 KB	250 GB/day; 30 days ≈ 7.5 TB before replication and transforms. Media dominates bytes.
Realtime heartbeat = 20,000 / 30 seconds	667 heartbeat cycles/s globally, plus inbound acknowledgments and message fan-out.
AI = 5% of 8M inbound × 1,500 input + 250 output tokens	400,000 calls/day; 600M input and 100M output tokens/day. Budget and cache by tenant.

With a burst of 324 events/s per cell, a 30-minute processing outage creates about 583,200 events of backlog. If the recovered cell sustains 800 events/s while arrivals remain at 324/s, net drain is 476/s and recovery takes about 20.4 minutes. If arrivals exceed capacity, adding time will not clear the queue.

A starting load-test configuration might use 24 ingress partitions per cell, replication factor 3, minimum in-sync replicas 2, and producers requiring all configured acknowledgments. Partition count must be validated against hot keys and consumer cost. Size databases, workers, network throughput, and connection pools for one-zone loss and skew; uniform tenant distribution is only a simplifying assumption. [8]

Monthly cost = compute + transactional stores + broker/workflow services + media storage and egress + observability + backups + AI + provider messages + support operations. Provider cost is delivered eligible messages by market/category/rate period; AI cost is model-specific token usage. Obtain dated regional quotes and run a load test before assigning a dollar budget. Dedicated cells and cross-region recovery create a material fixed baseline.

24 Failure handling and disaster recovery

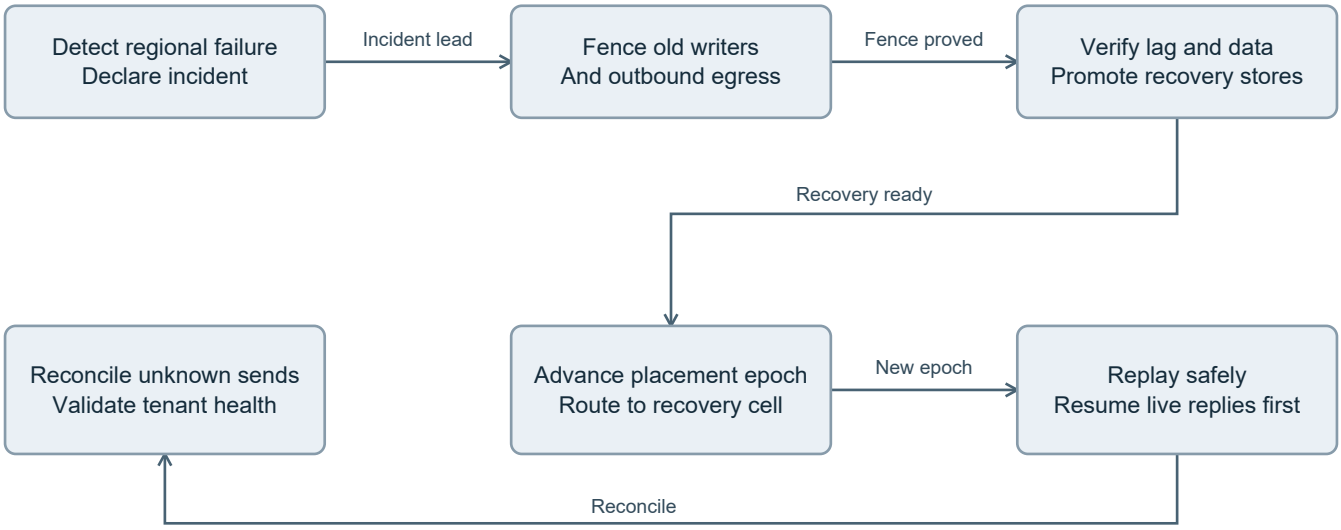


Figure 11. Promotion requires verified fencing. A region partition must not produce two active senders for one tenant.

Failure	Required behavior
Database or broker unavailable	Do not acknowledge unpersisted work. Keep committed backlog; fail readiness and alert.
Worker crashes mid-transaction	Transaction rolls back or commits atomically; redelivery is deduplicated.
Provider times out after transmission	Mark outcome unknown; reconcile evidence; do not automatically duplicate the send.
AI or CRM outage	Circuit breaker, bounded retries, explicit fallback or handoff; webhook acceptance continues.
Zone loss	Healthy replicas accept work; redistribute within reserved headroom; protect quorum.
Regional loss or network partition	Freeze bulk work, fence old writers, restore/promote stores, advance epoch, resume and reconcile.

Replicate database backups/WAL, event archives, object data, configuration, secrets references, and workflow history to an approved recovery region. Define replication and recovery independently for each store. Durable workflow state needs a supported replication or restore mechanism; rebuilding timers from an inbox snapshot is not equivalent. Redis presence can be discarded and rebuilt.

A global lease or fencing authority must prevent stale-region sends after promotion. Old dispatchers fail closed when their short lease expires; network controls or revoked regional credentials provide a second fence. If the old region cannot be proven fenced, postpone outbound promotion even if this exceeds RTO. Asynchronous replication can lose up to the measured lag, including recent dedupe state; quarantine affected send windows for reconciliation.

Run quarterly recovery drills and regular backup restores. Reapply deletion tombstones, verify record counts and checksums, compare provider observations, and test access controls before reopening all traffic. Failback is a controlled migration, never an automatic DNS flip while both regions can write.

25 Delivery pipeline and deployment

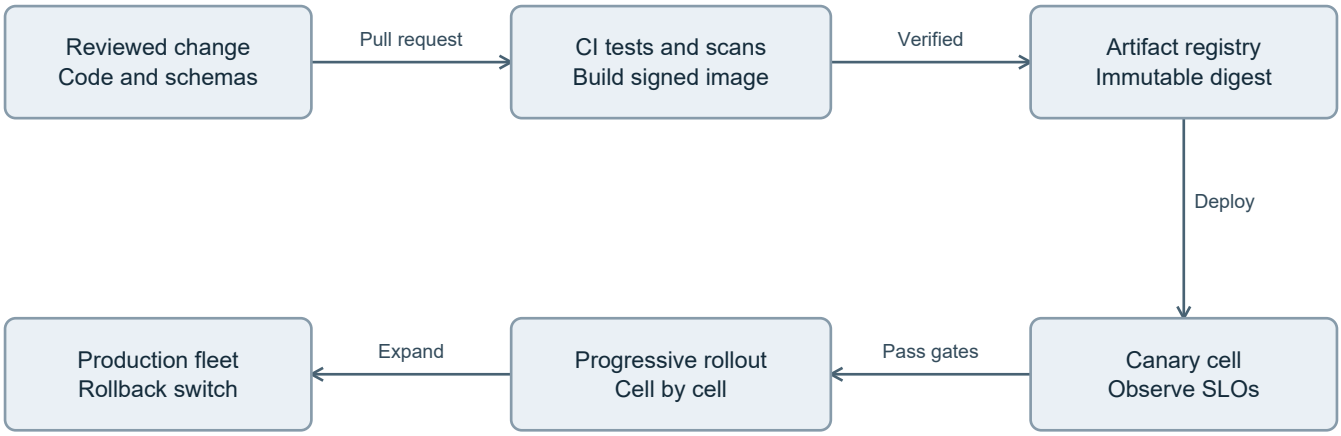


Figure 12. Infrastructure and application changes pass through the same reviewed release process; cells limit the blast radius.

Reference stack

Use React and TypeScript for the web console, Go for ingress/dispatch and high-throughput workers, and a consistent typed service framework for business APIs. Avoid adding languages without an operational reason. Use managed Kubernetes for the enterprise deployment, managed PostgreSQL and Redis, managed Kafka, object storage, a durable workflow engine, and OpenTelemetry collectors. A smaller runtime such as managed containers can replace Kubernetes if it meets isolation and rollout requirements.

On AWS, the concrete mapping is Route 53, CloudFront/WAF, regional load balancers, EKS, RDS or Aurora PostgreSQL, ElastiCache, MSK, S3, KMS, Secrets Manager, and a controlled egress path. Select exact editions and regions through measured requirements; these are service choices, not a price or feature guarantee. Keep account policies, network routes, IAM, schemas, and alarms in infrastructure as code.

Safe change management

CI runs unit tests, schema compatibility, API contracts, tenant isolation tests, migration checks, dependency/license/secret scans, and signed-image production. Staging uses a provider sandbox or controlled test number, realistic sanitized loads, and separate credentials. Deployment promotes an immutable image digest; no operator rebuilds production from a laptop.

Use expand-and-contract migrations: add compatible schema, deploy readers/writers that handle both forms, backfill in bounded batches, and remove old fields only after all consumers have migrated. Back up before destructive migrations and test rollback assumptions. A binary rollback cannot undo an incompatible database deletion.

Canary on an internal tenant or low-risk cell, compare queue age, errors, latency, and message correctness, then expand. Version workflow workers and keep existing executions compatible with their recorded history. Drain long-lived connections and workers gracefully. Separate feature flags for AI auto-send, new flow versions, provider API upgrades, and campaign admission.

26 Verification and launch acceptance

Test scenario	Pass condition
Webhook duplicates and mixed batches	One canonical message per provider ID; all valid items persisted; repeat delivery does not repeat bot actions.
Invalid signature or forged tenant context	No domain state change; event rejected and safe security evidence recorded.
Crash after DB commit before publish	Outbox resumes and downstream processes exactly one logical effect after dedupe.
Crash after provider accepts but before local result	Intent becomes unknown or reconciled; no blind second send.
Status events arrive out of order	Read/delivery projection remains consistent; raw observations are retained.
Agent takes over while AI is running	Bot generation is invalidated; unsent intent is blocked by ownership fence.
Opt-out after audience snapshot	All not-yet-dispatched promotional intents are blocked or cancelled.
One hot tenant sends a huge campaign	Other tenants meet live-reply SLO; quotas and backlog controls activate.
Tenant ID swapped in API, job, search or AI query	No cross-tenant data, signed URL, action, or existence disclosure.
Workflow publish during active session	Existing session stays on old version; new session uses intended release.
Worker restart during timer and HTTP action	Wait resumes once logically; side effect uses stable action ID.
Zone loss and regional promotion	Measured RPO/RTO meet target; no two active senders; unknown window reconciled.
Deletion then backup restore	Tombstones reapplied across all stores; deleted content is not exposed.
Provider API version upgrade	Contract suite and real test send/receive pass for every supported message type.

Run sustained capacity, burst, soak, queue-recovery, and failure-injection tests. Model hot conversations, skewed tenants, large attachments, slow clients, token expiry, invalid templates, expired messaging windows, and provider throttling. Include campaigns concurrent with live support rather than testing each in isolation.

Launch evidence must include a restore report, load-test report, tenant-isolation review, security findings with remediation, provider onboarding approval, alert-to-runbook drill, and signed operational ownership. Measure AI against a curated business question set and adversarial retrieval content; approve auto-send separately from assisted drafting.

27 Enterprise implementation sequence

Enterprise from day one describes the launch quality bar, not a requirement to release every feature at once. Build a narrow end-to-end path first on the final tenancy, durability, and security foundations, then expand capabilities. The ranges below are planning estimates for an experienced team; procurement and provider approvals can dominate elapsed time.

Phase	Deliverable and exit gate	Planning range
A Foundations	Cloud accounts, network cells, identity, tenant model, CI, observability, secrets, threat model and provider approval work started.	Weeks 1 to 4
B Core messaging	Onboarding, durable webhook ingestion, conversation store, outbound intents, policy checks, receipts, basic inbox and recovery tests.	Weeks 4 to 10
C Automation and campaigns	Flow compiler/runtime, handoff, audience snapshots, fair quotas, templates, consent evidence, connector framework.	Weeks 9 to 16
D Enterprise operations	SSO/SCIM, dedicated-cell provisioning, audit exports, deletion, billing reconciliation, capacity tests, regional recovery drill.	Weeks 14 to 22
E AI and controlled launch	Knowledge ingestion, retrieval evaluation, tool controls, assisted replies, staged auto-send, pilot customers and security review.	Weeks 19 to 26

Ownership and staffing

A credible initial team is approximately 12 to 16 people: a technical lead/architect; four or five backend engineers; two frontend engineers; two platform/SRE engineers; one security engineer; one test/automation engineer; one AI/data engineer; and product/design coverage. Some roles may be shared, but the work still needs named owners. A 24/7 response commitment requires an adequately staffed rotation and support plan beyond two SREs.

The platform team owns cells, networking, secrets, CI, backup, and recovery. Messaging owns channel adapters, ingestion, delivery, and policy. Experience owns inbox, flow editor, and tenant administration. Automation/AI owns runtime correctness and evaluations. Security and finance review authorization, audit, and billing controls. Every production service has an on-call owner and a runbook.

Release boundary

The first enterprise release can offer WhatsApp, a shared inbox, reliable campaigns, approved connector actions, and deterministic flows while keeping AI in draft mode. Add channels through the adapter contract after testing their distinct permissions, delivery semantics, limits, and feature sets. Do not flatten unsupported channel features into a misleading universal message API.

28 Decisions to finalize before engineering

Decision	Recommended default	What changes the choice
Provider relationship	Official Cloud API with tenant-owned WABAs; direct provider billing where supported	Partner eligibility, billing responsibilities, customer account model.
Cloud and launch regions	One cloud; tenant home regions with approved recovery pairing	Residency, procurement, latency, service availability and contracts.
Tenancy	Shared bounded cells with dedicated-cell option	Customer isolation requirements, size and economics.
Regional writes	One active write region per tenant	A verified business need for global multi-writer state, with added complexity.
Data and execution	PostgreSQL + Kafka + durable workflows + object storage	Measured throughput and managed-service availability.
Message dispatch	One gateway; explicit unknown state after ambiguous send	A provider-supported idempotent send API may simplify recovery.
AI launch mode	Draft or handoff until business evaluations pass	Measured quality, action sensitivity, approved models and data policies.
Retention and recovery	Contract-driven policy with tested deletion and restores	Legal holds, regulated data, regional replication constraints.
Service promises	Publish only measured SLAs and support commitments	Load tests, recovery drills, on-call staffing and provider dependencies.

Engineering invariants

• Every tenant-owned read, write, event, cache entry, object, and external action has an enforced tenant boundary.

• An accepted webhook and an accepted send command each have durable evidence before acknowledgment.

• Every retryable action has a stable identity; every ambiguous external action has an explicit reconciliation path.

• Only the currently authorized owner can initiate a new reply, and only the outbound gateway can send it.

• Campaign throughput, AI usage, and customer integrations cannot exhaust reserved live-messaging capacity.

• A tenant has one active writer region; recovery never enables two independent senders.

• Published flow versions, policy decisions, and administrative changes remain traceable for the required retention period.

The product will feel seamless when these guarantees are visible through clear state: queued, sending, delivered, waiting for input, assigned to an agent, blocked by policy, or needing recovery. Hiding uncertainty creates duplicate messages and confusing support cases. Design the user interface around the actual distributed-system state.

Sources and implementation references

Research checked on 29 September 2026. Public product pages establish feature scope; official technical references support protocol and infrastructure choices. All service boundaries, capacity examples, SLOs, retention defaults, and rollout estimates are design proposals. The references below do not establish WhatChimp's internal technology stack.

1 WhatChimp product overview

https://whatchimp.com/

Public feature scope: campaigns, inbox, AI and integrations.

2 WhatChimp chatbot guide

https://help.whatchimp.com/docs/bot-manager/build-a-complete-whatsapp-chatbot-step-by-step

Public visual-flow behavior; reference for feature parity.

3 Meta official WhatsApp API workspace

https://www.postman.com/meta/whatsapp-business-platform/overview

Cloud API, Business Management, WhatsApp Flows and Embedded Signup overview.

4 Meta Cloud API collection overview

https://www.postman.com/meta/whatsapp-business-platform/collection/wlk6lh4/whatsapp-cloud-api

Account assets, permissions and access-token concepts. Confirm selected onboarding path.

5 Meta Messages API collection

https://www.postman.com/meta/whatsapp-business-platform/folder/13382743-ba8d099d-007e-4b52-b9f2-3cf3c60e4fbc

Provider message endpoint and supported message categories.

6 Meta webhook verification reference

https://whatsapp.github.io/WhatsApp-Nodejs-SDK/api-reference/webhooks/start/

GET challenge and POST signature behavior. The SDK is archived; use as protocol background, not a runtime dependency.

7 AWS transactional outbox pattern

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

Reliable publication of database state changes and the need for consumer idempotency.

8 Apache Kafka design documentation

https://kafka.apache.org/41/design/design/

Partition ordering, replication and producer durability concepts.

Further references and validation notes

9 WhatsApp Business Platform pricing

https://whatsappbusiness.com/products/platform-pricing/

Delivered-message pricing, categories, market dependence and customer-service-window explanation. Use current account rate cards at launch.

10 Temporal execution architecture

https://github.com/temporalio/temporal/blob/main/docs/architecture/README.md

Durable workflows, deterministic execution and idempotent activities. See also https://docs.temporal.io/tasks.

11 Kubernetes Network Policies

https://kubernetes.io/docs/concepts/services-networking/network-policies/

Network isolation, ingress and egress policies and implementation requirements.

12 OWASP SSRF prevention guidance

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

Destination restrictions and defensive URL-fetching controls.

13 PostgreSQL row security policies

https://www.postgresql.org/docs/18/ddl-rowsecurity.html

Policy enforcement and owner, superuser and BYPASSRLS exceptions.

Provider details to validate during implementation

Some Meta developer pages were login- or rate-limited during research. Official Meta-maintained API collections and the public WhatsApp pricing page provide the accessible foundation. Before implementing, confirm the current Graph API version, Embedded Signup eligibility and scopes, webhook event schemas and retry horizon, per-number throughput, coexistence support, template rules, and WhatsApp Flows encryption requirements against the live developer portal.

Do not copy marketing claims about unlimited broadcasting, zero ban risk, or fixed approval times into service guarantees. Account restrictions and delivery remain subject to provider policy, account quality, recipient behavior, and available capacity. Price schedules can change independently of your application release, so the architecture keeps them effective-dated and configurable.

Glossary

CDC change data capture. DLQ dead-letter queue. HMAC keyed message authentication code. IAM identity and access management. mTLS mutual TLS. OIDC OpenID Connect. RLS row-level security. SCIM identity provisioning protocol. SLO service-level objective. SSRF server-side request forgery. WAF web application firewall. WSS WebSocket over TLS.